

When the model says 0.9, is it bad 90% of the time?

Last lecture measured how well our cheese detector *ranks* batches (ROC, PR). But it also outputs a **number** — $\hat{p} = 0.9$. If we tell the factory “90% chance this batch is bad,” is that *true*?

A model can **rank** perfectly (high AUC) and still output probabilities that are systematically wrong — saying 0.9 when only 70% of such batches are actually bad. That gap is **(mis)calibration**.

This deck: what calibration is, how to measure it, and how to fix it.

Outline

What calibration is

Why it matters

Fixing it

In practice

What “calibrated” means

A classifier is **calibrated** if its probabilities match reality:

Among all batches it scores $\hat{p} \approx 0.8$, about **80%** are actually bad.

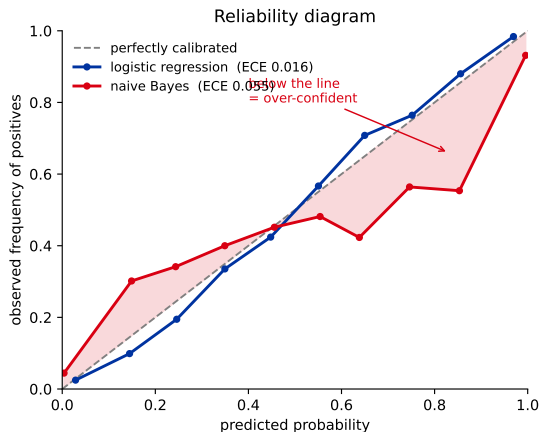
- ▶ A **score** ranks; a **calibrated probability** can be read as a real chance.
- ▶ Calibration is about the *value* of $\hat{p}$, not the *order* — a separate property from accuracy or AUC.
- ▶ We check it by bucketing predictions and comparing predicted vs. observed frequency.

The reliability diagram

Bin predictions by $\hat{p}$; plot **predicted** (x) vs. **observed frequency** (y).

- ▶ On the diagonal = perfectly calibrated.
- ▶ **Below** the line = **over-confident** (predicts higher than reality).
- ▶ Above = under-confident.

Logistic regression tracks the diagonal; an over-confident model (here naive Bayes) bows *below* the line, pushing $\hat{p}$ toward the extremes (ECE 0.06 vs. 0.02).

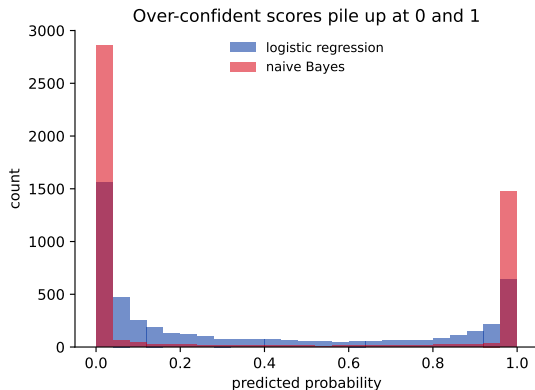


Over-confidence, seen another way

Same two models, histogram of their predicted probabilities.

- ▶ **Over-confident** models pile scores near 0 and 1 — they rarely say “maybe”.
- ▶ Calibrated models spread mass across $[0, 1]$, matching real uncertainty.

Piling at the extremes is the fingerprint of miscalibration — and exactly what breaks any decision that reads $\hat{p}$ as a chance.



Calibration $\neq$ ranking

A model can order batches perfectly yet report wrong probabilities. AUC won't catch it — and any cost decision that uses $\hat{p}$ will be off.

Predict-first: perfect ranking, honest probabilities?

A cheese detector ranks **perfectly** — every bad batch scores above every clean one, so **ROC AUC** = 1.0. Now **halve every predicted probability** ($\hat{p} \rightarrow \hat{p}/2$).

Predict: what happens to the AUC? to the calibration?

- ▶ **AUC: unchanged** (= 1.0). Halving preserves the *order*, and AUC sees only order.
- ▶ **Calibration: wrecked**. Batches now scored 0.45 are still bad 90% of the time — the numbers stopped meaning anything.

Ranking and calibration are different properties. A model can order perfectly and still lie about probabilities — and AUC will never catch it.

Calibration is separate from discrimination

- ▶ **Discrimination** (ROC AUC): does the score *order* positives above negatives?
Cares only about ranking.
- ▶ **Calibration**: are the score *values* true frequencies?

As the last slide showed, a model can reach **AUC** = 1 (perfect ranking) and still be badly miscalibrated — ranking and probability values are different properties.

Why it matters. The cost-optimal threshold $c^* = \frac{C_{FP}}{C_{FN} + C_{FP}}$ and *any* expected-value decision assume $\hat{p}$ is a true probability. Reporting “90% risk” to the factory (or a doctor) only means something if it is calibrated.

Who is miscalibrated?

Usually fine

- ▶ **Logistic regression** — it minimizes log-loss, a *proper scoring rule* (one minimized only by reporting the true probabilities, so honesty is rewarded).

Over-confident (scores too extreme)

- ▶ Naive Bayes (independence assumption)
- ▶ Boosted trees, deep neural nets
- ▶ SVMs (margins, not probabilities)

Under-confident (scores too timid)

- ▶ Random forests (averaging pulls toward 0.5)

Watch out: `class_weight` and resampling (for imbalance) also distort probabilities — they change the base rate the model trains on, so re-calibrate afterwards.

Measuring calibration

- ▶ **Reliability diagram** — the picture (qualitative).
- ▶ **ECE** (expected calibration error): bin predictions, average $|\text{observed} - \text{predicted}|$ weighted by bin size. The headline number; 0 is perfect. *Caveat: depends on the binning.*
- ▶ **Brier score** $= \frac{1}{n} \sum (\hat{p}_i - y_i)^2$ — a proper scoring rule, but it *mixes* calibration and ranking, so a sharp model can score well while miscalibrated.
- ▶ **Log-loss** — also proper; punishes confident mistakes hardest.

For a *pure* calibration number use **ECE** (or read the reliability diagram); Brier and log-loss are good overall scores but conflate calibration with discrimination.

Worked example: computing ECE by hand

Bin the predictions; in each bin compare the **average prediction** $\hat{p}_b$ to the **observed frequency** of bad batches, then average the gaps *weighted by bin size*.

Weight each bin's gap by its share of all predictions:

bin	n_b	$\hat{p}_b$ / obs.	gap
[0.0, 0.2)	50	0.10/0.10	0.00
[0.2, 0.4)	20	0.30/0.25	0.05
[0.6, 0.8)	20	0.70/0.50	0.20
[0.8, 1.0]	10	0.90/0.60	0.30

$n = 100$ predictions; empty bins contribute 0.

$$\begin{aligned}\text{ECE} &= \sum_b \frac{n_b}{n} |\text{obs}_b - \hat{p}_b| \\ &= \frac{50}{100}(0) + \frac{20}{100}(.05) \\ &\quad + \frac{20}{100}(.20) + \frac{10}{100}(.30) \\ &= .01 + .04 + .03 = \mathbf{0.08}.\end{aligned}$$

The two high-score bins are **over-confident** (say 0.7/0.9, only 0.5/0.6 actually bad) and drive almost all the error. *ECE is just a weighted average of the reliability diagram's vertical gaps.*

Fixing calibration

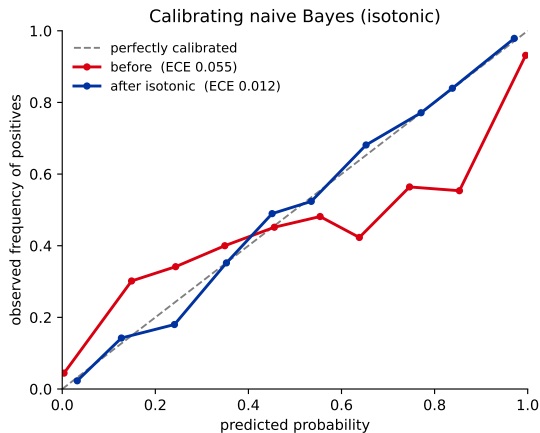
Don't retrain — learn a small post-hoc map from the model's raw scores to true probabilities, fit on held-out data.

Post-hoc calibration: Platt and isotonic

Learn a map from raw scores to true probabilities, on **held-out** data:

- **Platt scaling**: fit a sigmoid $\sigma(as + b)$. Two parameters — good for *small* data; assumes an S-shaped distortion.
- **Isotonic regression**: a free *monotonic* step function. More flexible — needs *more* data (overfits when small).

Isotonic snaps naive Bayes onto the diagonal: ECE 0.06 $\rightarrow$ 0.01.



Calibration in sklearn

```
from sklearn.calibration import CalibratedClassifierCV

# wrap any classifier; calibrate by cross-validation on the training
# data
cal = CalibratedClassifierCV(
    GaussianNB(),
    method="isotonic",      # or "sigmoid" (Platt) for small data
    cv=5,
)
cal.fit(X_train, y_train)
cal.predict_proba(X_test)[:, 1]      # now-calibrated probabilities
```

- ▶ method="sigmoid" (Platt) for small data; "isotonic" for larger ($\gtrsim$ a few thousand).
- ▶ Calibrate on data the base model did **not** train on — cv handles this.
- ▶ `CalibrationDisplay.from_estimator(...)` draws the reliability diagram.

When does calibration matter?

Calibrate when...

- ▶ you *report* probabilities (risk to a doctor / factory);
- ▶ you make **cost / expected-value** decisions or tune a threshold by cost;
- ▶ you **average or ensemble** probabilities from several models.

Don't bother when...

- ▶ you only need a **ranking** (ROC AUC, top- k % / lift);
- ▶ the threshold is tuned directly on the metric you care about.

Golden rule: fit calibration on data the base model never saw, and don't re-tune the threshold on that same split — both bias the result optimistically.

Recap

- ▶ **Calibration** = predicted probabilities match observed frequencies (read it off the **reliability diagram**).
- ▶ It is **separate from ranking**: high AUC does not imply good probabilities.
- ▶ Measure with **ECE** (pure calibration) or Brier / log-loss (overall, proper scores).
- ▶ Logistic regression is usually calibrated; naive Bayes, boosted trees and nets are over-confident; random forests under-confident; imbalance fixes distort probabilities.
- ▶ Fix post-hoc with **Platt** (sigmoid, small data) or **isotonic** (flexible, more data) via `CalibratedClassifierCV` — always on held-out data.

Next: imbalanced learning — class weights and resampling when the positive class is rare (and why they then need re-calibration).